Service Overview

ภาพรวมของบริการ (Service Overview)

AlertResponse Service

1. Service Owner

นายกิตติธัช เด่นสกุลประเสริฐ รหัสนักศึกษา 6609650079 ภาคพิเศษ

2. Service Purpose

AlertResponse Service เป็นบริการที่รับผิดชอบการสร้างและเผยแพร่การแจ้งเตือนภัยไปยังประชาชนในพื้นที่เป้าหมายผ่าน SMS และรวบรวมการตอบกลับของประชาชน (รับทราบ / ปลอดภัย / ต้องการความช่วยเหลือ) เพื่อสร้างภาพรวมสถานการณ์ระดับพื้นที่สำหรับการตัดสินใจของหน่วยงานที่เกี่ยวข้อง

ถ้าไม่มี AlertResponse Service หน่วยงานจะไม่สามารถทราบได้ว่าใครได้รับข้อมูลแล้ว และใครต้องการความช่วยเหลือเร่งด่วน ส่งผลให้การจัดสรรทรัพยากรล่าช้า

3. Pain Point ที่แก้ไข

เมื่อเกิดภัยพิบัติ ข้อมูลสถานการณ์มักกระจายหลายช่องทางและเปลี่ยนแปลงรวดเร็ว ทำให้ประชาชนสับสน และหน่วยงานไม่สามารถทราบได้อย่างชัดเจนว่าประชาชนในพื้นที่ได้รับข้อมูลแล้วหรือยัง และมีใครบ้างที่ต้องการความช่วยเหลืออย่างเร่งด่วน

3. Target Users

- ประชาชนในพื้นที่เสี่ยง (รับ SMS และตอบกลับสถานะ)
- หน่วยงานภาครัฐที่ติดตามสถานการณ์ / ศูนย์ควบคุม

4. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)
    - สร้างและจัดเก็บข้อมูลการแจ้งเตือน (Alert Master Data)
    - ใช้ข้อมูลตำแหน่งจาก Incident Service เพื่อกำหนดกลุ่มเป้าหมาย
    - ส่งข้อความแจ้งเตือนผ่าน SMS Gateway
    - รับและประมวลผล SMS ตอบกลับ
    - คำนวณสถิติภาพรวม (total_sent, total_acknowledged, total_need_help)
    - บังคับใช้รูปแบบการตอบกลับที่ถูกต้อง (business validation)

- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
    - การจัดการข้อมูลเหตุการณ์ภัยพิบัติหลัก (Incident Master Data)
    - การจัดทีมช่วยเหลือหรือ dispatch หน่วยกู้ภัย
    - การจัดการศูนย์พักพิง
    - การเก็บข้อมูลส่วนบุคคลเชิงลึกของประชาชน
    - การกำหนดแผนอพยพ

5. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การเปลี่ยนสถานะของ alert (CREATED (ตอนสร้าง)⟶ ACTIVE (ส่ง SMS สำเร็จและรอคนตอบ) ⟶ CLOSED (ปิดรับ response)⟶ FAILED)
- การสรุปจำนวนผู้ตอบรับแต่ละประเภท
- การ validate รูปแบบการตอบกลับจากประชาชน
- การปฏิเสธ response ที่ไม่ถูกต้อง
- การ retry การส่ง SMS เมื่อ gateway ล้มเหลว

การตัดสินใจอิงจาก:

- citizen_response (1=acknowledged, 2=safe, 3=need_help)
- สถานะของ alert ปัจจุบัน
- จำนวน response ที่ได้รับ
- business rules ที่กำหนด

บริการสามารถตัดสินใจได้เองภายใต้ business rules ที่กำหนด <u>โดยไม่ต้องรอ/ต้องรอ</u>การอนุมัติจากมนุษย์ในกรณีปกติ

- หาก response ไม่ตรง format ⟶ ปฏิเสธทันที
- หาก alert อยู่ในสถานะ closed ⟶ ไม่รับ response ใหม่
- หาก SMS ส่งไม่สำเร็จ ⟶ retry อัตโนมัติไม่เกิน 3 ครั้ง

6. Owned Data

- Alert Master Data
  ข้อมูลพื้นฐานของการแจ้งเตือน เช่น ประเภทของภัยพิบัติ เนื้อหาข้อความ คำแนะนำในการปฏิบัติตัว และพื้นที่เป้าหมาย ข้อมูลส่วนนี้เป็นแกนหลักของ domain การแจ้งเตือน และ ถูกใช้โดยตรงในการสื่อสารกับประชาชน ดังนั้นบริการนี้ควรเป็นผู้ดูแลข้อมูลดังกล่าวเพื่อควบคุมความถูกต้องและความสอดคล้องของเนื้อหา
- Alert Operational State
  ข้อมูลสถานะของการแจ้งเตือน เช่น สถานะปัจจุบัน วันที่สร้าง และวันที่เผยแพร่ ข้อมูลนี้ใช้ควบคุมลำดับขั้นตอนการทำงานของระบบ และกำหนดว่า alert อยู่ในช่วงใดของ lifecycle จึงต้องอยู่ภายใต้การดูแลของบริการนี้
- Response Summary State
  ข้อมูลสรุปผลการตอบกลับของประชาชน เช่น จำนวนผู้ที่ตอบรับ จำนวนผู้ปลอดภัย และจำนวนผู้ที่ต้องการความช่วยเหลือ ข้อมูลนี้เกิดจากการประมวลผลภายในระบบ และใช้สนับสนุนการติดตามสถานการณ์ จึงควรเป็นความรับผิดชอบของบริการนี้

7. Linked Data (Reference Only)

ก่อนเผยแพร่หรือปิดการแจ้งเตือน ระบบจะตรวจสอบสถานะของเหตุการณ์จาก Incident Service เพื่อให้มั่นใจว่า incident ยังอยู่ในสถานะที่เหมาะสม (เช่น ACTIVE)

- incident_id
- สถานะของ incident (ACTIVE / RESOLVED)

บริการนี้ใช้ข้อมูลดังกล่าวเพื่อประกอบการตัดสินใจ และไม่จัดเก็บหรือเป็นเจ้าของข้อมูล incident อย่างถาวร

8. Non-Functional Requirements

Reliability

- Retry การส่ง SMS ไม่เกิน 3 ครั้ง พร้อม backoff
- ระบบต้องไม่ crash หากได้รับ SMS ผิดรูปแบบ
- ต้องรองรับ concurrent response โดยไม่ทำให้ count ผิดพลาด

Data Integrity

- ต้องมี idempotency สำหรับ response เพื่อป้องกันการนับซ้ำ
- ต้อง validate input ทุกครั้งก่อนบันทึก
- สถิติ summary ต้องสอดคล้องกับจำนวน response จริง

Security

- API ต้องมี authentication

Observability

- ต้องมี logging สำหรับทุก SMS send attempt
- ต้องมี trace id สำหรับแต่ละ alert

Sync Contract

Synchronous Function Contract

Base URL: http://api.alertresponse.com

API Contract #1: Create Alert

ข้อมูลทั่วไป

- **Name:** Create Alert
- **Method:** POST
- **Path:** /alerts
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับสร้างรายการแจ้งเตือนโดยอ้างอิงจากเหตุการณ์ (incident) ที่มีอยู่ใน Incident Service

1. ตรวจสอบความถูกต้องของข้อมูล
2. ตรวจสอบสถานะ incident จาก Incident Service (ต้องเป็น ACTIVE)
3. ดึงข้อมูล incident (incidentType, severity, location)
4. บันทึก snapshot ของ incident ภายใน AlertResponse Service
5. Publish AlertCreated event เพื่อให้กระบวนการส่ง SMS ทำงานแบบ asynchronous

Request

Path/Query Params

- ไม่มี

Headers

- Content-Type: application/json
- Authorization: Bearer <access_token>
- Idempotency-Key: <uuid>

Body:

```
{
    "incidentId": "INC-20260218-001",
    "message": "เกิดเหตุน้ำท่วม โปรดอพยพไปยังจุดปลอดภัย"
}
```

Validation Rules

- incidentId ต้องมีอยู่ใน Incident Service
- incident.status ต้องเป็น ACTIVE
- message ต้องไม่ว่าง
- ต้องไม่มี alert ที่ ACTIVE อยู่แล้วสำหรับ incident เดียวกัน

Response

Success: 201 Created

```
{
    "alertId": "ALT-20260218-001",
    "status": "CREATED"
}
```

Error:

- 400 Bad Request – ข้อมูลไม่ถูกต้องหรือไม่ครบถ้วน
- 401 Unauthorized – ไม่มีสิทธิ์ใช้งาน
- 404 Not Found – ไม่พบ incidentId
- 409 Conflict – มี alert ที่ active อยู่แล้ว
- 500 Internal Server Error – ระบบขัดข้อง

Dependency / Reliability

- พึ่งพาฐานข้อมูลหลัก (Database)
- Publish AlertCreated event ไปยัง Message Broker
- รองรับ idempotency เพื่อป้องกันการสร้าง Alert ซ้ำ
- ไม่รอการส่ง SMS (แยกเป็น asynchronous process)

API Contract #2: Record Citizen Response

ข้อมูลทั่วไป

- **Name:** Record Citizen Response
- **Method:** POST
- **Path:** /alerts/{alertId}/responses
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับบันทึกการตอบกลับของประชาชนผ่าน SMS Gateway โดยรองรับการตอบกลับแบบ
รองรับการตอบกลับประเภท:

- ACK (รับทราบ)
- SAFE (ปลอดภัย)
- HELP (ต้องการความช่วยเหลือ)

ระบบจะ validate และบันทึกข้อมูลลงฐานข้อมูลทันที

Request

Path/Query Param

- alertId (Path Parameter) – รหัสแจ้งเตือนที่ประชาชนตอบกลับ

Headers

- Content-Type: application/json
- Idempotency-Key: <uuid>

Body

```
{
    "phoneNumber": "0812345678",
    "responseType": "HELP",
    "helpType": "INJURED",
    "peopleCount": 2,
    "addressText": "99/12 หมู่ 4 ต.สุเทพ"
}
```

Validation

- alertId ต้องมีอยู่ในระบบ
- alert.status ต้องไม่เป็น CLOSED
- responseType ∈ {ACK, SAFE, HELP}
- หาก responseType = HELP ⟶ ต้องระบุ helpType และ peopleCount
- helpType ∈ {TRAPPED, INJURED, FOOD_WATER}
- peopleCount ≥ 1
- addressText ต้องไม่ว่างเมื่อเป็น HELP
- phoneNumber ต้องอยู่ในรูปแบบที่ถูกต้อง
- ต้องรองรับ idempotency เพื่อป้องกันการบันทึกซ้ำจาก SMS retry

Response

Success: 200 OK

```
{
    "responseId": "RES-20260218-015",
    "status": "RECORDED"
}
```

Error:

- 400 Bad Request – รูปแบบข้อมูลไม่ถูกต้อง
- 404 Not Found – ไม่พบ alertId
- 409 Conflict – มีการบันทึกซ้ำ
- 500 Internal Server Error – ระบบขัดข้อง

Dependency / Reliability

- พึ่งพาฐานข้อมูลหลัก (Database)
- ใช้ transaction เพื่อป้องกันข้อมูลไม่สมบูรณ์
- อัปเดต summary counters แบบ atomic operation
- รองรับ retry จาก SMS Gateway กรณี network timeout
- รองรับ idempotency เพื่อป้องกันการนับซ้ำ

Async Contract

Asynchronous Function Contract

Message Contract #1: AlertCreated

ข้อมูลทั่วไป

- **Message Name:** AlertCreated
- **Interaction Style:** Event-Driven (Publish/Subscribe)
- **Producer:** AlertResponse Service
- **Consumer:** SMS Dispatcher Service
- **Channel/Queue:** alert-created-topic
- **Version:** v1

คำอธิบาย

เหตุการณ์นี้ถูก publish เมื่อ Alert ถูกสร้างสำเร็จและอยู่ในสถานะ ACTIVE

SMS Dispatcher Service จะ subscribe event นี้ และดำเนินการ broadcast ข้อความแจ้งเตือนไปยังประชาชนในพื้นที่ที่ได้รับผลกระทบผ่าน Operator-based Geo SMS (production) หรือ Mock SMS Gateway (testing)

เลือกใช้ asynchronous messaging เพราะการ broadcast SMS เป็นงานที่ใช้เวลานานและไม่ควร block API หลัก การแยกเป็น event-driven ช่วยให้ระบบตอบกลับ client ได้รวดเร็วและเพิ่ม reliability เมื่อเกิด failure

Request

Message Headers

- messageId (UUID)
- timestamp (เวลา publish (ISO-8601))
- eventType = AlertCreated
- version = v1
- source = AlertResponseService

Message Body

```
{
    "alertId": "ALT-20260218-001",
    "incidentId": "INC-20260218-001",
    "incidentType": "FLOOD",
    "severity": "HIGH",
    "location": {
```

"type": "Polygon",
"coordinates": [...]
},
"message": "เกิดเหตุน้ำท่วม โปรดอพยพไปยังจุดปลอดภัย",
"createdAt": "2026-02-18T10:15:30Z"
}

Field	Type	Required	Description
alertId	String	yes	รหัสแจ้งเตือน
incidentId	String	yes	รหัสเหตุการณ์ที่อ้างอิง
incidentType	Object (GeoJSON)	yes	ประเภทเหตุการณ์ (snapshot)
severity	String	yes	ระดับความรุนแรง (snapshot)
location	String	yes	พื้นที่ได้รับผลกระทบ (snapshot)
message	String	yes	ข้อความแจ้งเตือนที่จะส่ง
createdAt	String(DateTime)	yes	เวลาสร้าง alert

Field Definition:

Validation Rules

1. alertId ต้องไม่ว่าง
2. severity $\in$ {LOW, MODERATE, HIGH, CRITICAL}
3. location ต้องอยู่ในรูปแบบ GeoJSON ที่ถูกต้อง
4. message ต้องมีความยาวไม่เกิน 500 ตัวอักษร
5. createdAt ต้องอยู่ในรูปแบบ ISO-8601

Response

Message Headers

- messageId
- timestamp
- eventType = SMSDispatchResult
- source = SMSDispatcherService

Success Message Body


```
{
    "alertId": "ALT-20260218-001",
    "status": "SMS_DISPATCHED",
    "processedAt": "2026-02-18T10:16:10Z",
    "failureReason": null
}
```


Reject/Error Message Body


```
{
    "alertId": "ALT-20260218-001",
    "status": "FAILED",
    "processedAt": "2026-02-18T10:16:10Z",
    "failureReason": "SMS Gateway Unavailable"
}
```


Field Definition:

Field	Type	Required	Description
alertId	String	yes	รหัสแจ้งเตือน
status	String	yes	สถานะการส่ง
processedAt	DateTime	yes	เวลาที่ประมวลผล
failureReason	String	No	เหตุผลหากล้มเหลว

Validation Rules

1. status $\in$ {SMS_DISPATCHED, FAILED}
2. หาก status = FAILED $\longrightarrow$ ต้องมี failureReason
3. ต้องมี retry อย่างน้อย 3 ครั้งก่อนส่งไป Dead Letter Queue (DLQ)

Service Data

Service Data

1) Alert Master Data (Owned by this service)

Field Name	Type	Required	Description	Example
alertId	string	Y (Primary Key)	รหัสแจ้งเตือน	ALT-001
incidentId	string	Y (Reference Only)	อ้างอิง Incident จาก Incident Service	INC-789
message	string	Y	ข้อความแจ้งเตือนที่ถูก generate แล้ว	น้ำท่วมระดับสูง โปรดอพยพ
severitySnapshot	enum	Y	เก็บ severity ตอนสร้าง (low/moderate/high/critical)	HIGH
locationSnapshot	Object (GeoJSON)	Y	พื้นที่ได้รับผลกระทบ ณ เวลาสร้าง Alert	{ "type": "Polygon", ... }
status	enum	Y	CREATED / ACTIVE / CLOSED / FAILED	ACTIVE
totalRecipients	integer	Y	จำนวนผู้รับที่ส่งแจ้งเตือน	1250
totalAcknowledged	integer	Y	จำนวนผู้ตอบ ACK	800
totalSafe	integer	Y	จำนวนผู้ตอบ SAFE	350
totalNeedHelp	integer	Y	จำนวนผู้ตอบ HELP	100
createdBy	string	N	ผู้สร้าง (เช่น system / admin-user)	system
createdAt	datetime	Y	เวลาที่สร้าง	2026-02-18T10:20:00Z
updatedAt	datetime	Y	เวลาที่แก้ไขล่าสุด	เวลาที่แก้ไขล่าสุด
closedAt	datetime	N	เวลาที่ปิดรับ response	2026-02-18T12:00:00Z

severitySnapshot เพราะ Incident severity อาจเปลี่ยนภายหลัง

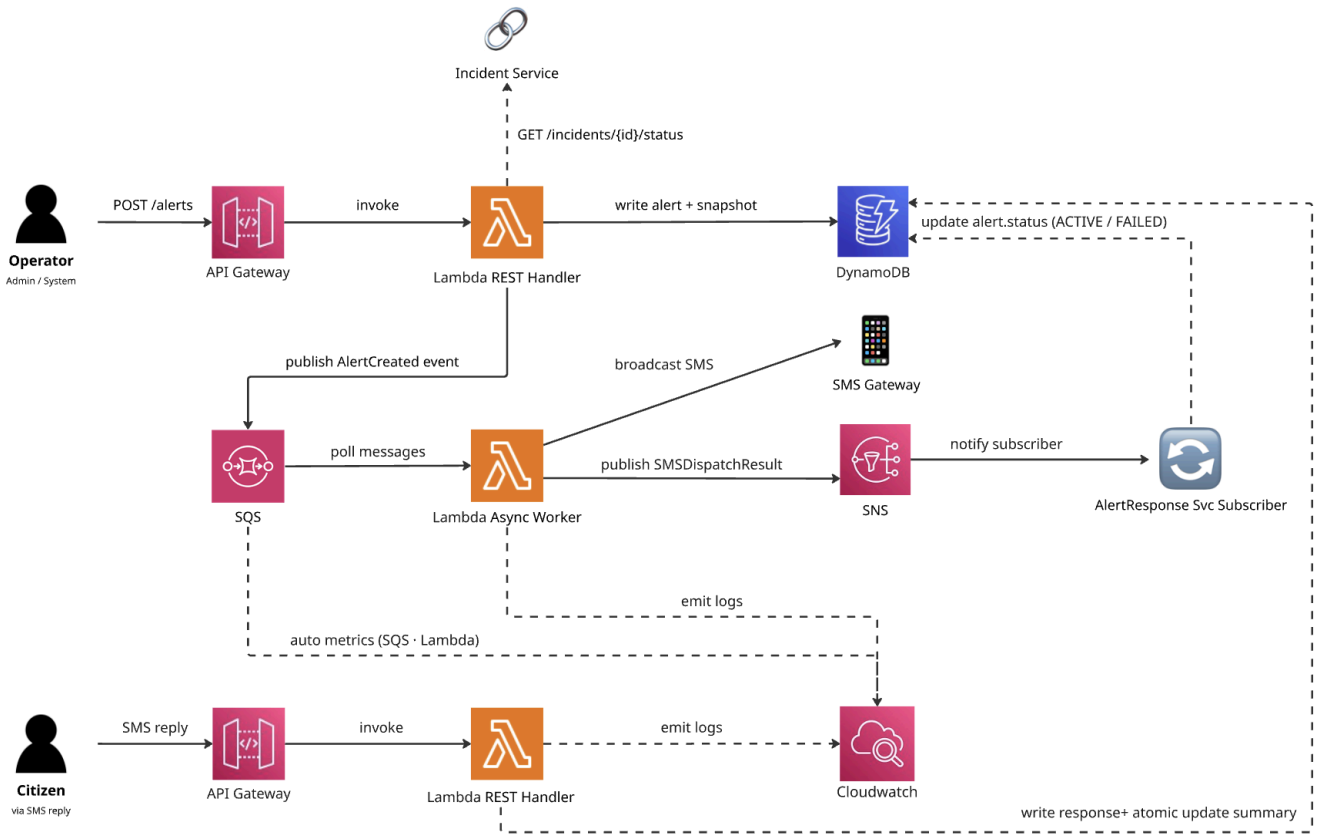
ถ้าไม่ snapshot ประวัติ alert จะไม่ตรงกับเหตุการณ์ตอนส่งจริง

2) Citizen Response Data (Owned by this service)

Field Name	Type	Required	Description	Example
responseid	string	Y (Primary Key)	รหัสการตอบกลับ	RES-015
alertId	string	Y (Foreign Key)	อ้างอิง Alert	ALT-001
incidentId	string	Y (Reference Only)	อ้างอิง Incident	INC-789
phoneNumber	string	Y	เบอร์ผู้ตอบ	0812345678
responseType	enum	Y	ACK / SAFE / HELP	HELP
helpType	enum	N (Required if HELP)	TRAPPED / INJURED / FOOD_WATER	INJURED
peopleCount	integer	N (Required if HELP)	จำนวนผู้ได้รับผลกระทบ	2
addressText	string	N (Required if HELP)	ที่อยู่ที่ขอความช่วยเหลือ	99/12 หมู่ 4
createdAt	datetime	Y	เวลาที่ตอบกลับ	2026-02-18T10:40:12Z

Service Architecture

Service Architecture



Components (องค์ประกอบของระบบ)

1. ฝั่งผู้ใช้งาน (Users)

- **Operator (Admin/System):** เจ้าหน้าที่ที่เรียกใช้งาน REST API เพื่อสร้างการแจ้งเตือนภัยและดึงข้อมูลสรุปผล
- **Citizen:** ประชาชนในพื้นที่เสี่ยงภัยที่รับข้อความและส่ง SMS ตอบกลับเพื่อแจ้งสถานะของตนเอง

2. จุดรับคำขอ (API Gateway)

- ทำหน้าที่เป็นประตูหน้าบ้านรับคำขอแบบ Synchronous ทั้งหมด ได้แก่:
  - `POST /alerts` สำหรับรับคำสั่งสร้างการแจ้งเตือน
  - `GET /alerts/{id}` สำหรับดึงข้อมูลสรุปสถิติ
  - `POST /alerts/{id}/responses` สำหรับรับข้อความ SMS ที่ประชาชนตอบกลับ

3. กลุ่มประมวลผลแบบทันที (Synchronous Lambda Handlers)

- **CreateAlertHandler:** รับคำสั่งสร้าง Alert -> วิ่งไปเช็คข้อมูลกับ Service ของเพื่อน -> บันทึกตารางลง DB -> โยนงานเข้าคิว SQS แล้วตอบกลับ `201 Created` ทันทีโดยไม่ต้องรอส่ง SMS เสร็จ
- **GetAlertSummaryHandler:** รับคำขอจาก Operator เพื่อไปดึงยอดสรุปสถิติ (คนรับทราบ/ปลอดภัย/ขอความช่วยเหลือ) จากตาราง `AlertData` มาแสดงผล
- **RecordResponseHandler:** รับข้อความที่ประชาชนตอบกลับมา (เช่น 1, 2, 3) -> ตรวจสอบความถูกต้อง (Validation) -> อัปเดตยอด Counter ใน DB ทันที (Atomic Update) พร้อมเก็บประวัติการตอบกลับ

4. บริการของเพื่อนในชั้นเรียน (Classmate Services - External)

- **IncidentReporter Service:** บริการที่เราวิ่งไปถามแบบ Synchronous เพื่อดึงสถานะเหตุการณ์ (Status), ความรุนแรง (Severity) และประเภทภัยพิบัติ (Incident Type) มาประกอบการส่งข้อความ
- **IncidentImpactZone Service:** บริการที่เราวิ่งไปถามแบบ Synchronous เพื่อขอดึงพิกัดหรือรหัสพื้นที่ (Location) ที่ได้รับผลกระทบ เพื่อเอามากวาดหาเบอร์โทรประชาชน

5. กลุ่มประมวลผลเบื้องหลัง (Asynchronous Messaging & Workers)

- **Amazon SQS (Queue):** คิวรับพักงาน (Event: `AlertCreated`) เพื่อไม่ให้หน้าบ้านค้างระหว่างรอส่งข้อความ
- **SMSDispatchWorker (Lambda):** ดึงงานจาก SQS ไปไล่ส่ง SMS หาประชาชนตามพื้นที่ (มีระบบ Retry หากส่งไม่สำเร็จ) และเมื่อส่งครบจะประกาศผลลัพธ์ออกไปทาง SNS
- **Amazon SNS (Topic):** ช่องทางสำหรับประกาศผลลัพธ์การส่ง SMS (`SMSDispatchResult`)
- **AlertResponseSubscriber (Lambda):** คอยฟังเสียงจาก SNS พอได้ยินว่าส่ง SMS เสร็จแล้ว จะวิ่งไปเปลี่ยนสถานะ Alert ใน DynamoDB เป็น `ACTIVE` หรือ `FAILED`

6. ฐานข้อมูลและระบบมอนิเตอร์ (Storage & Monitoring)

- **DynamoDB:** จัดเก็บข้อมูลหลักของระบบ ได้แก่ `AlertData` (เก็บสถานะและยอดสรุป) และ `ResponseData` (เก็บประวัติการตอบกลับรายคน)
- **CloudWatch:** ระบบหลังบ้านสำหรับเก็บ Logs ของ Lambda ทุกตัว และคอยมอนิเตอร์สุขภาพของระบบ (Metrics)

Explanation (อธิบายการทำงานของระบบ)

สถาปัตยกรรมของ AlertResponse Service ถูกออกแบบมาเพื่อรองรับงานที่ต้องการความรวดเร็ว (Synchronous) และงานที่ต้องใช้เวลาประมวลผลนาน (Asynchronous) อย่างเป็นระบบครับ โดยมีเวิร์กโฟลว์การทำงานดังนี้:

1. การสร้างและกระจายการแจ้งเตือน (Create & Broadcast Flow) เมื่อเจ้าหน้าที่ (Operator) ต้องการประกาศเตือนภัย จะส่งคำสั่งผ่าน API Gateway เข้ามาที่ `CreateAlertHandler` ระบบจะไม่ทำงานแบบโดดเดี่ยว แต่จะวิ่งไปดึงข้อมูลความรุนแรงจาก **IncidentReporter Service** และดึงขอบเขตพื้นที่จาก **IncidentImpactZone Service** (ซึ่งเป็น Service ของเพื่อน) เพื่อความสมบูรณ์ของข้อมูล

จากนั้นระบบจะบันทึกข้อมูลลง DynamoDB โยนคำสั่งส่ง SMS เข้าคิว **Amazon SQS** และตอบกลับ `201 Created` ให้เจ้าหน้าที่ทันทีโดยไม่ปล่อยให้ระบบรอนาน

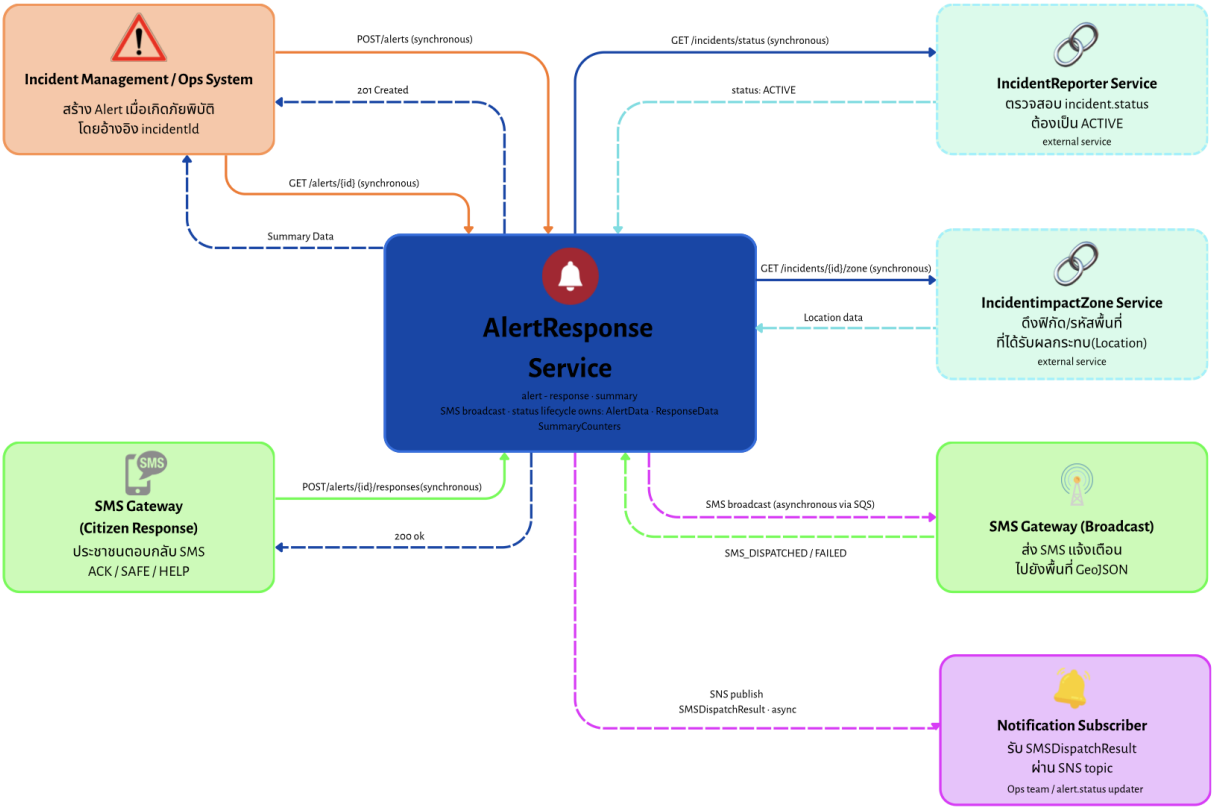
หลังจากนั้น **SMSDispatchWorker** จะมารับช่วงต่อโดยดึงงานจาก SQS ไปไล่ส่ง SMS แบบเบื้องหลัง (Asynchronous) เมื่อส่งเสร็จสิ้น มันจะประกาศผลลัพธ์ผ่าน **Amazon SNS** ซึ่งจะมี **AlertResponseSubscriber** คอยดักฟังอยู่ เพื่อรับหน้าที่ไปอัปเดตสถานะสุดท้าย (ACTIVE/FAILED) ลงใน DynamoDB การออกแบบแนวนี้ช่วยลดภาระการทำงาน (Decoupling) และทำให้ระบบทนทานขึ้นครับ

2. การโต้ตอบกับประชาชน (Citizen Response Flow) เมื่อประชาชนได้รับ SMS และพิมพ์ตอบกลับ (เช่น 1,2,3) ข้อความจะวิ่งผ่าน API Gateway มาที่ `RecordResponseHandler` ระบบจะทำการตรวจสอบความถูกต้องของข้อมูล (Validation) และทำการอัปเดตยอดสถิติแบบ Atomic Update ลงใน DynamoDB ทันที เพื่อให้ข้อมูลเป็นปัจจุบันที่สุด

3. การติดตามผลลัพธ์ (Monitoring Flow) ในระหว่างที่เหตุการณ์กำลังดำเนินอยู่ เจ้าหน้าที่สามารถเรียก API ไปที่ `GetAlertSummaryHandler` เพื่อดึงยอดสรุป (เช่น มีคนปลอดภัยกี่คน, ต้องการความช่วยเหลือกี่คน) ไปแสดงบน Dashboard ได้ทันที นอกจากนี้ การทำงานของระบบทั้งหมดจะถูกส่งไปบันทึก Logs และ Metrics ที่ **Amazon CloudWatch** เพื่อให้มั่นใจว่าระบบสามารถทำงานได้อย่างเสถียรในสถานการณ์ฉุกเฉินครับ

Service Interaction

Service Interaction



Service Interaction (การโต้ตอบระหว่างบริการ)

ระบบ AlertResponse Service ของเราทำงานเป็นตัวกลางที่ต้องเชื่อมต่อกับหลายๆ องค์ประกอบครับ โดยสามารถแบ่งทิศทางการสื่อสารออกเป็น 2 ฝั่งหลักๆ คือ **Upstream** (ฝั่งที่เรียกเข้ามาหาเรา) และ **Downstream** (ฝั่งที่เราเรียกออกไป) ดังนี้ครับ:

Upstream Services (บริการต้นทางที่เรียกเข้ามาหาระบบเรา)

เป็นกลุ่มที่เรียกใช้งานเข้ามาที่ API ของเราโดยตรง ซึ่งเราออกแบบให้ตอบกลับทันที (Synchronous) เพื่อประสบการณ์ใช้งานที่ดีครับ

1. **ระบบจัดการของเจ้าหน้าที่ (Operator / Admin UI):**
    - เข้ามาเรียกใช้ API แบบ **Synchronous** ผ่าน `POST /alerts` เพื่อสั่งสร้างการแจ้งเตือนภัย ระบบเราจะรับเรื่องและตอบสถานะ `201 Created` กลับไปให้ทันทีโดยไม่ต้องรอส่ง SMS จนเสร็จ เพื่อป้องกันไม่ให้ค้าง
    - นอกจากนี้ยังเรียก `GET /alerts/{id}` แบบ **Synchronous** เพื่อดึงตัวเลขสถิติแบบ Real-time (ยอดคนปลอดภัย/ขอความช่วยเหลือ) ไปแสดงผลบน Dashboard
2. **SMS Gateway (ช่องทางรับข้อความจากประชาชน):**
    - เมื่อประชาชนในพื้นที่เสี่ยงพิมพ์ SMS ตอบกลับมา Gateway จะยิง Request เข้ามาที่ `POST /alerts/{id}/responses`แบบ **Synchronous** เพื่อให้ระบบเราตรวจสอบข้อมูล และทำการนับยอดสรุป (Atomic Update) ลงฐานข้อมูลทันที

Downstream Services (บริการปลายทางที่เราเรียกใช้งานหรือส่งงานต่อ)

ส่วนนี้คือจุดที่ระบบของเราต้องไปดึงข้อมูลหรือโยนงานไปให้คนอื่นทำต่อ **ซึ่งครอบคลุมการเชื่อมต่อกับ Service ของเพื่อนร่วมชั้นเรียน (Classmate Services) ถึง 2 บริการครับ**:

1. **IncidentReporter Service (Classmate Service):**
    - ทันทีที่มีคำสั่งสร้างการแจ้งเตือน ระบบเราจะวิ่งไปเรียก API ของเพื่อนตัวนี้แบบ **Synchronous** เพื่อดึงข้อมูล "สถานะ (Status)", "ความรุนแรง (Severity)" และ "ประเภทภัยพิบัติ (Incident Type)" มาประกอบร่างเป็นข้อความแจ้งเตือนที่สมบูรณ์และแม่นยำ
2. **IncidentImpactZone Service (Classmate Service):**
    - ในจังหวะเดียวกัน เราก็วิ่งไปเรียก API ของเพื่อนตัวนี้แบบ **Synchronous** ด้วย เพื่อขอดู "พิกัดหรือรหัสพื้นที่ (Location)" ที่กำลังเกิดภัยพิบัติ ข้อมูลนี้สำคัญมากเพราะเราต้องใช้เพื่อระบุขอบเขตว่าต้องดึงเบอร์โทรศัพท์ของประชาชนในโซนไหนบ้างมาทำการส่ง SMS
3. **SMS Gateway (ช่องทางกระจายข้อความเตือนภัย):**
    - เนื่องจากงานส่ง SMS ทีละหลายพันเบอร์ใช้เวลานาน ระบบเราจึงไม่ทำเองแบบรอผล แต่ใช้วิธีโยนงานเข้าคิว SQS แล้วให้ Lambda Worker ทยอยส่งคำสั่งไปยัง SMS Gateway แบบ **Asynchronous** (ทำงานอยู่เบื้องหลัง) ซึ่งตัว Worker มีระบบดักจับข้อผิดพลาดและ Retry ให้รวดอัตโนมัติหากเครือข่ายมือถือมีปัญหา
4. **ระบบประกาศผลแจ้งเตือน (ผ่าน Amazon SNS):**
    - เมื่อ Worker กระจาย SMS เสร็จสิ้น จะไม่ไปยุ่งกับ Database เอง แต่จะประกาศผลลัพธ์ออกไปทาง SNS Topic แบบ **Asynchronous** เพื่อให้ Subscriber ที่รอฟังอยู่มารับช่วงต่อไปอัปเดตสถานะของ Alert ว่าสำเร็จ (ACTIVE) หรือล้มเหลว (FAILED)

Summary (สรุปภาพรวมการโต้ตอบ)

ระบบของเรามีการผสมผสานระหว่างการทำงานแบบ Sync และ Async ได้เลือกใช้ **Synchronous** ในจุดที่ต้องการความถูกต้องและข้อมูลที่เป็นปัจจุบันทันที (เช่น การตรวจสอบพิกัดความรุนแรงจาก Service ของเพื่อน และการรับการโต้ตอบขอความช่วยเหลือจากประชาชน) ในขณะเดียวกัน เราเลือกใช้ **Asynchronous (SQS/SNS)** ในงานที่ใช้เวลานานและกินทรัพยากรสูง (เช่น การกระจาย SMS จำนวนมาก) เพื่อทำ Decoupling ลดภาระของระบบหลัก ทำให้ระบบสามารถสเกลรองรับสถานการณ์ฉุกเฉินที่มีโหลดสูงๆ ได้อย่างมั่นคงครับ

Dependency Mapping

Dependency Mapping – AlertResponse Service

1. **IncidentReporter Service (Classmate Service)**

- Type: Service
- Interaction Style: Synchronous (REST)
- Purpose: ตรวจสอบว่าเหตุการณ์ยังมีสถานะเป็น ACTIVE อยู่หรือไม่ และดึงข้อมูลระดับความรุนแรง (Severity) รวมถึงประเภทภัยพิบัติ (Incident Type) เพื่อนำมาประกอบร่างข้อความแจ้งเตือนภัย
- Criticality: Critical
- Failure Handling: * หาก IncidentReporter Service ไม่ตอบสนอง หรือส่งข้อมูลกลับมาผิดพลาด ระบบจะปฏิเสธคำขอสร้าง Alert ทันที พร้อมตอบกลับ HTTP 500/503 ให้ Operator ทราบเพื่อหลีกเลี่ยงการส่งข้อความแจ้งเตือนที่ไม่สมบูรณ์

2. IncidentImpactZone Service (Classmate Service)

- Type: Service
- Interaction Style: Synchronous (REST)
- Purpose: ดึงข้อมูลพิกัดหรือรหัสพื้นที่ (Location / Area Code) ที่ได้รับผลกระทบ เพื่อนำมาเป็นเงื่อนไขในการดึงเบอร์โทรศัพท์ของประชาชนเป้าหมายที่จะต้องส่ง SMS ไปเตือน
- Criticality: Critical
- Failure Handling: * หาก IncidentImpactZone Service ขัดข้อง ระบบจะไม่สามารถระบุพื้นที่เป้าหมายได้ จะทำการปฏิเสธคำขอสร้าง Alert และตอบกลับ error ไปยัง Operator ทันที

3. Amazon SQS (alert-created-queue)

- Type: Queue
- Interaction Style: Asynchronous (Event/Job Queue)
- Purpose: เป็นคิวรับคำสั่งสร้าง Alert (AlertCreated) เพื่อพักงานและส่งต่อให้ Worker นำไปดำเนินการกระจาย SMS เบื้องหลัง ช่วยไม่ให้ API ฝั่งหน้าบ้าน (Operator) ต้องรอจนกว่าจะส่ง SMS เสร็จ
- Criticality: Critical
- Failure Handling: * หาก Lambda Worker ดึงคิวไปประมวลผลแล้วล้มเหลว (เช่น SMS Gateway พังชั่วคราว) SQS จะใช้กลไก Retry อัตโนมัติ (เช่น 3 ครั้ง) หากยังล้มเหลวซ้ำอีก ข้อความคำสั่งนั้นจะถูกส่งไปยัง Dead Letter Queue (DLQ) เพื่อให้ Admin เข้ามาตรวจสอบและแก้ไขภายหลัง

4. SMS Gateway (External Service)

- Type: External Service
- Interaction Style: Synchronous (API Call) *ถูกเรียกผ่าน Asynchronous Worker*

- Purpose: บริการภายนอกที่ใช้สำหรับส่งข้อความ SMS แจ้งเตือนภัยไปยังโทรศัพท์มือถือของประชาชนแบบรายบุคคล (Broadcast)
- Criticality: Critical
- Failure Handling: * หาก SMS Gateway ขัดข้องหรือไม่สามารถส่งข้อความได้ Lambda Worker จะทำการ Retry ตามระยะเวลาที่กำหนด หากเกินขีดจำกัดจะถือว่าการกระจายข้อความรอบนั้น FAILED และจะส่งผลลัพธ์ความล้มเหลวนี้ไปทาง SNS

5. Amazon SNS (dispatch-result-topic)

- Type: Topic
- Interaction Style: Asynchronous (Event Notification)
- Purpose: ช่องทางโทรโข่งสำหรับประกาศผลลัพธ์การส่ง SMS (SMSDispatchResult) ว่าสำเร็จลุล่วงกี่เบอร์ หรือล้มเหลว เพื่อให้ Subscriber รับช่วงต่อไปอัปเดตสถานะการแจ้งเตือน
- Criticality: Non-critical
- Failure Handling: * หาก Publish ข้อความลง SNS ไม่สำเร็จ ระบบจะบันทึก Error Log ลง CloudWatch (การส่ง SMS ไปหาประชาชนสำเร็จไปแล้ว เหตุการณ์นี้จะกระทบแค่สถานะหน้า Dashboard ของ Admin ที่อาจจะอัปเดตช้าลง)

6. Amazon DynamoDB

- Type: Database
- Interaction Style: Internal Data Access
- Purpose: จัดเก็บข้อมูลหลักของระบบ ได้แก่ `AlertData` (รายละเอียดการแจ้งเตือน, สถานะ, และยอดสรุปสถิติ) และ `ResponseData` (ประวัติการตอบกลับของประชาชน)
- Criticality: Critical
- Failure Handling: * กรณีสร้าง Alert: หาก Write ข้อมูลล้มเหลว จะไม่โยนคำสั่งเข้า SQS และตอบ Error กลับหา Operator
    - กรณีรับ SMS ตอบกลับ: หาก Update ยอดสรุปล้มเหลว (เช่น ตรวจพบเงื่อนไข ConditionExpression ว่า Alert ถูกปิดหรือไม่ได้ ACTIVE แล้ว) ระบบจะป้องกันไม่ให้บันทึกข้อมูลซ้ำซ้อน และส่ง Error (HTTP 400) แจ้งกลับไป

Github

นาย กิตติธัช เด่นสกุลประเสริฐ 6609650079
Github https://github.com/Kittithatdensakulprasert/Alertresponse_service